内部系统接入 ChatGPT
MCP + OAuth 只读插件制作教程

从零到一：把任何没有公开 API 的内部业务系统，安全地变成 ChatGPT 可用的只读工具

全文脱敏 · 可公开分享 · 实战验证通过

0. 我们要做出什么效果

完成本教程后，你将拥有一个这样的能力：

- 在 ChatGPT 对话框里直接用自然语言查询内部业务系统的数据（例如：「查一下目前在招的某类职位」）；
- ChatGPT 通过标准 **OAuth 2.0** 登录授权后，只能调用你开放的 **5 个只读工具**，无法修改任何数据；
- 内部使用时，同一个服务保留另一个「全量工具」入口，供你自己的 Agent 使用，互不影响；
- 全部密钥只存在本机 `.env`，公网入口有密码门 + OAuth 双重保护。

本教程以「内部业务数据查询」为场景，方法同样适用于 CRM、工单、库存、知识库等任何内部系统。

1. 整体架构



核心思想一句话：**MCP 服务永远不直接暴露**——对内走 localhost，对外先过 Cloudflare 隧道，再按 URL 路径分流：给 ChatGPT 的入口强制 OAuth，给自己人的入口保留密码门。

2. 准备清单

需要什么	用途	费用

一台常驻开机的电脑（本教程用 Mac）	跑 MCP 服务和隧道	—
Node.js 18+	MCP Server 运行环境	免费
一个自有域名 + Cloudflare 账号	Tunnel 内网穿透，提供 HTTPS 公网入口	免费
Auth0 账号	OAuth 授权服务（签发/校验令牌）	免费档够用
ChatGPT Plus	开发者模式、创建自定义插件	订阅制
Caddy	反向代理与路径分流（也可用 Nginx 替代）	免费

3. 第一步：把内部系统封装成 MCP 服务

很多内部系统没有公开 API，只有网页后台。办法是：用代码模拟登录拿到会话 Cookie，然后把后台的查询接口（打开浏览器开发者工具，在网络面板里一个个实测出来）封装成标准 MCP 工具。

3.1 技术选型

- 官方 SDK：`@modelcontextprotocol/sdk`，传输方式选 **Streamable HTTP**（ChatGPT 要求 HTTP 可达）；
- 入参校验用 `zod`；HTTP 请求直接用 Node 内置 `fetch`，不引入多余依赖；
- 服务只监听 `127.0.0.1`，绝不直接监听公网。

3.2 关键设计：一套服务，两个「档案」（profile）

这是整个方案最重要的设计决策：

入口路径	给谁用	工具数量	认证方式
`/mcp`	自己的内部 Agent	全量（本例 88 个，含写操作）	Caddy 层 Basic 密码门
`/chatgpt/mcp`	ChatGPT 插件	**只读 5 个**	OAuth Bearer Token

只读 profile 注册的工具示例：

```
search_joborders      // 搜索业务单据（如职位）
get_joborder          // 查看单条单据详情
search_candidates     // 搜索业务对象（如候选人）
get_candidate         // 查看单个对象详情
get_candidate_resume  // 精读单个对象的完整档案
```

原则：给 AI 的工具描述（description）一律用中文写清「什么时候用、注意什么、是否修改数据」；返回数据要瘦身，只留关键字段，长文本截断，避免撑爆模型上下文。

4. 第二步：用 Auth0 搭 OAuth 授权服务

ChatGPT 官方推荐用标准 OAuth 保护自定义 MCP 服务。我们不去自己实现授权服务器（复杂且易错），直接用 Auth0 免费档。

4.1 创建 API

1. Auth0 后台 → **Applications → APIs → Create API**；
2. Identifier（受众 audience）填你的公网域名，例如 `https://mcp.example.com` ——**必须和后面 MCP 服务校验的 audience 完全一致**；
3. 签名算法选 **RS256**（非对称，公钥验签，服务端不存密钥）。

4.2 定义权限（scope）

在 API 的 Permissions 里添加一个最小权限：`data.read`。**只加这一个**，不开放任何写权限。

4.3 两个容易漏掉的开关

① 设置 Tenant Default Audience： Auth0 → Settings → General，把 Default Audience 填成你的 API Identifier。不设的话，签发的令牌可能不带 `aud` 字段，服务端会拒绝。
② 允许第三方应用的默认授权： API → Settings，找到第三方应用（Third-Party）的 User-delegated 授权，勾选 `data.read`。ChatGPT 通过动态注册创建的客户端属于第三方应用，没有这条授权会在同意页报 `access_denied`。

5. 第三步：给 MCP 服务加 OAuth 门禁

在 `/chatgpt/mcp` 路径上加应用层校验，三件事：

5.1 公布「受保护资源元数据」

ChatGPT 会先来探路。实现 `/.well-known/oauth-protected-resource`，返回 JSON：

```
{
  "resource": "https://mcp.example.com",
  "authorization_servers": ["https://your-tenant.us.auth0.com/"],
  "scopes_supported": ["data.read"]
}
```

5.2 无令牌一律 401，并给出「质询」

```
HTTP/1.1 401 Unauthorized
WWW-Authenticate: Bearer resource_metadata="https://mcp.example.com/.well-known/oauth-protected
                  scope="data.read", error="invalid_token"
```

ChatGPT 读到这个头，就知道该去哪个授权服务器、申请什么权限。

5.3 有令牌则严格验签

用 Auth0 的 JWKS 公钥（`https://your-tenant.us.auth0.com/.well-known/jwks.json`）校验 JWT，逐项检查：

- `iss` 必须是你的 Auth0 租户地址；
- `aud` 必须等于你的资源域名；
- `exp` 未过期；`scope` 必须包含 `data.read`。

安全要点：校验失败只返回 401，日志里**绝不打印完整令牌**；内部系统的账号密码等真实凭据只放 `.env`，并确认 `.gitignore` 已忽略。

6. 第四步：安全地暴露到公网

6.1 Cloudflare Tunnel

用 `cloudflared` 建一条命名隧道（免费），把自有域名的一个子域名（如 `mcp.example.com`）指向本机 Caddy。好处：不需要公网 IP、不用开端口、自带 HTTPS。用系统服务（launchd / systemd）托管，进程挂了自动拉起。

6.2 Caddy 路由分流（顺序是关键）

```
mcp.example.com {
    # 只读 OAuth 入口，给 ChatGPT —— 必须放在兜底规则之前
    handle /chatgpt/* {
        reverse_proxy 127.0.0.1:39200
    }
    # 其余路径：保留原有 Basic 密码门，给内部使用
    basic_auth {
        用户名 哈希后的密码
    }
    reverse_proxy 127.0.0.1:3080
}
```

改完执行 `caddy validate` 校验，再 `caddy reload` 热加载。

6.3 上线前回归清单（全部通过才算完）

检查项	预期
本机 `/mcp`	200，全量工具在线
公网 `/.well-known/oauth-protected-resource`	200
公网 `/chatgpt/mcp` 无令牌	401，且带 Bearer 质询头
公网 `/mcp` 无密码	401，仍是 Basic 质询（密码门没被绕开）
服务错误日志	0 行

7. 第五步：在 ChatGPT 创建连接

1. **开启开发者模式**：ChatGPT → 设置 → 账户安全与登录 → 开发人员模式（会标「高风险」，自己用可接受）；
2. 打开 `chatgpt.com/plugins` → **创建应用**；
3. 填写：名称（如「业务数据助手」）、描述（写清「仅提供只读访问」）、服务器 URL 填 **只读入口** `https://mcp.example.com/chatgpt/mcp`；
4. 身份验证选 **OAuth**（不要选「混合」）；
5. 展开**高级 OAuth 设置**：注册方法选 **动态客户端注册（DCR）**——让 ChatGPT 自己去 Auth0 注册客户端，不用手抄 Client ID/Secret；
6. 确认自动发现的端点（Auth URL / 令牌 URL / 注册 URL / 资源）都来自你的 Auth0 和域名，默认作用域勾选 `data.read`；
7. 点创建 → 插件设置页点**连接** → 弹出登录窗口 → **本人登录并同意授权**；
8. 回到插件设置页，核对「操作」列表：**必须恰好是预期的 5 个只读工具**，每个都标「读取」、所需范围 `data.read`。

验收红线：如果工具列表出现几十个工具或任何写工具，说明 URL 填错（指到了全量入口）或只读 profile 失效——立刻停用排查，不要继续测试。

8. 实战中踩过的三个坑（排错实录）

坑 1：创建时报「dynamic client registration is disabled」

现象：ChatGPT 点创建，红色报错 `Dynamic client registration failed: 400`。

原因：Auth0 租户默认关闭 DCR。

解决：Auth0 → Settings → Advanced → 打开 **Dynamic Client Registration (DCR)**。连接建立后可以再关回去——已注册的客户端不受影响。

坑 2：授权页报「no connections enabled for the client」

现象：跳到登录页直接报 `invalid_request`，连登录框都不显示。

原因：DCR 注册出来的是「第三方应用」，Auth0 规定第三方应用只能使用**域级（domain-level）连接**，默认一个都没有。

解决：Auth0 → Authentication → Database → 你的数据库连接 → Settings，打开 **Promote Connection to Domain Level** 并保存。提升后所有第三方应用自动获得该连接。

坑 3：登录页能打开了，但没有账号可登

现象：Auth0 租户是新建的，User Management 里一个用户都没有。

解决：最省事的是把 **Google 社交连接同样提升为域级**，登录页点「Continue with Google」一键登录，然后同意授权即可。也可以在 User Management 手工创建一个邮箱用户。

通用排错顺序：① 先 curl 元数据端点是否 200 → ② 无令牌请求是否 401 且带 Bearer 质询 → ③ 看 Auth0 后台 Monitoring → Logs 的原始错误 → ④ 令牌问题只看脱敏 claims（iss/aud/exp/scope），不打印完整令牌。

9. 安全验收清单（上线后逐条过）

#	检查	标准
1	工具数量	ChatGPT 里恰好显示预期的只读工具，一个不多
2	负向测试	让 AI「加备注 / 删除记录 / 导出全部数据」，应明确拒绝并说明只读
3	无令牌访问	公网直连只读入口返回 401
4	旧入口未失守	全量入口仍然要密码，未因新路由被绕过
5	密钥管理	凭据只在 `.env`；代码、日志、聊天记录中无密码、无令牌
6	最小权限	Auth0 里只有 `data.read` 一个 scope，不写、不删

7	真实调用	在 ChatGPT 里真实跑一次查询，数据能正确返回

10. 一页纸复用 Checklist

阶段	动作
封装	实测内部系统接口 → MCP 工具化 → 拆「全量 / 只读」两个 profile 两个路径
授权	Auth0 建 API（audience=域名）→ 加只读 scope → 设 Default Audience → 配第三方默认授权
门禁	实现元数据端点 + 401 质询 + JWKS 验签（iss/aud/exp/scope）
公网	Cloudflare 隧道 → Caddy 分流（新路由放兜底前）→ validate + reload → 全量回归
连接	ChatGPT 开发者模式 → 创建应用（OAuth + DCR）→ 本人登录授权 → 核对工具清单
兜底	DCR 开关、域级连接、Google 登录三个坑的解法备查